AI Structural-Break Detector for Statistical Arbitrage

Software Engineering Architecture & Repository Specification

Postgraduate group project - Track A (Tech Team)

> **Architecture decision.** The course build uses a modular Python monolith: Streamlit imports a thin service layer directly. There is no FastAPI or microservice boundary. Batch jobs are the only authoritative writers to persistent analytical storage; the app is primarily read-only and presentation-focused.

Purpose: define the concrete codebase, repository, runtime boundaries, storage access patterns, configuration system, orchestration jobs, testing strategy, CI workflow, application integration, and deployment conventions needed to implement the previously specified data, econometric, ML, agent, and backtesting layers.

Scope boundary: this document does not repeat the statistical models, feature formulas, agent reasoning rules, data-source selection, or backtest methodology already specified elsewhere. It defines how those components are assembled into a maintainable, reproducible software system.

Document map and engineering scope

Section	Engineering question answered
1	What architectural style are we implementing, and why?
2	What runs where at runtime?
3	What is the exact repository tree?
4	What does each Python package own?
5	Which packages may depend on which others?
6	How are configuration and secrets managed?
7	How are files, Parquet, DuckDB, documents, and artifacts physically organised?
8	How is DuckDB initialised, versioned, and accessed safely?
9	How are model, prompt, policy, and run artifacts versioned?
10	Which batch jobs exist and what are their dependencies?
11	What is the service layer between domain code and Streamlit?
12	How is the Streamlit application structured without embedding business logic?
13	How are caching and performance handled?
14	How are logging, errors, and run status handled?
15	How are unit, integration, leakage, regression, and UI tests organised?
16	What does CI run and what must never depend on external APIs?
17	How does the team collaborate safely in Git/GitHub?
18	How is the demo/deployment packaged and operated?
19	What build order and acceptance gates should Codex follow?

Deliberate non-duplication. The data architecture has already defined what data exists and where it originates; the component PDFs have already defined finance, econometrics, ML, agents and backtesting. This specification focuses on code organization and system integration.

1. Architectural style

A modular Python monolith with direct Streamlit integration

1.1 Chosen pattern

The entire research system is one Python project and one deployable application. Component boundaries are enforced by packages and service interfaces, not by HTTP services. Streamlit calls Python service functions directly; those services orchestrate domain packages and query persistent storage.

Layer	Responsibility	Examples
Presentation	Render pages, tables, charts, filters and status. No finance/ML logic.	Streamlit pages, Plotly charts, badges, forms.
Application / services	Compose domain operations into use cases for the UI and scripts.	Scanner query, pair detail, investigation, strategy lab, monitor.
Domain / research	Finance, econometrics, ML, NLP, agents, controller, backtesting.	Relationship estimation, feature building, XGBoost inference, agent graph.
Infrastructure	Configuration, storage, files, database access, logging, artifact registry.	DuckDB connection, Parquet store, document store, model loader.
Persistent state	Versioned data and generated artifacts.	Parquet, DuckDB, documents, model files, manifests, MLflow.

1.2 Why no FastAPI

- **No network boundary inside the course application.** Python-to-Python calls are simpler, faster, easier to debug, and easier to deploy.
- **No duplicated interface layer.** Pydantic contracts can still validate service inputs/outputs without HTTP serialization.
- **No microservice pretense.** All components share one release cycle, one repository, one dataset, and one deployment target.
- **Extensibility remains possible.** A future API can wrap the service layer without rewriting the domain packages.

> **Core rule.** The Streamlit UI may call services, but it must not directly implement econometric estimation, feature engineering, model training, agent prompts, decision policy, or backtest logic.

2. Runtime architecture

How the application executes

```
User browser
   |
   v
Streamlit presentation layer
   | direct Python calls
   v
Application service layer
   |
   +--> read/query services --> DuckDB views/tables --> Parquet / metadata
   +--> model service -------> versioned model artifacts
   +--> investigation -------> retrieval + LangGraph agents
   +--> strategy lab --------> precomputed backtest artifacts

Offline/batch jobs (authoritative writers)
   ingestion -> relationships -> events -> features -> train -> agents -> backtest
      |           |           |          |       |       |
      +-------------+-----------+----------+--------+-------> persistent storage
```

2.1 Two execution modes

Mode	Purpose	Write policy
Research / batch	Build and refresh datasets, models, agent outputs and backtests.	May write Parquet, DuckDB, artifacts and cache files.
Interactive app	Browse results, inspect pairs/events, run bounded presentation workflows.	Read-only for canonical analytics. Optional ad-hoc agent results stay in a cache, not canonical tables.

2.2 Important operational decision

Canonical DuckDB analytics tables are treated as batch-produced research outputs. The app should not mutate them during ordinary browsing. This keeps displayed results reproducible and avoids ambiguous concurrent-write behaviour in a multi-session Streamlit deployment.

- Precompute headline historical results before the demo.
- Allow the UI to request fresh read-only queries freely.
- If an on-demand agent investigation is demonstrated, cache the result separately by event/source/prompt/model version and promote it to canonical analytics only through an explicit batch job.
- Never retrain a model or rebuild the historical dataset from a Streamlit button in the public demo.

3. Repository layout

Exact project skeleton

```
structural-break-detector/
|-- README.md
|-- pyproject.toml
|-- uv.lock
|-- .python-version
|-- .env.example
|-- .gitignore
|-- configs/
|   |-- universe.yaml
|   |-- data.yaml
|   |-- econometrics.yaml
|   |-- ml.yaml
|   |-- agents.yaml
|   |-- decision.yaml
|   |-- backtest.yaml
|   `-- app.yaml
|-- src/structural_break/
|   |-- config/
|   |-- data/
|   |-- storage/
|   |-- pairs/
|   |-- econometrics/
|   |-- events/
|   |-- features/
|   |-- ml/
|   |-- intelligence/
|   |-- agents/
|   |-- decision/
|   |-- backtest/
|   |-- services/
|   `-- common/
|-- app/
|   |-- Home.py
|   |-- pages/
|   `-- components/
|-- scripts/
|-- sql/
|-- data/
|-- artifacts/
|-- tests/
|-- docs/
`-- .github/workflows/ci.yml
```

Repository principle. Only source code, configuration templates, small deterministic fixtures, documentation, and version-control metadata belong in Git. Large datasets, model artifacts, MLflow runs, logs, caches, and secrets are excluded.

4. Package responsibilities

What each source package owns

Package	Owns	Must not own
config	Typed settings, paths, config loading, version/hash helpers.	Business logic or data retrieval.
data	Market/document ingestion, cleaning, calendar alignment, data contracts.	Econometric or model decisions.
storage	DuckDB, Parquet, document and artifact persistence helpers.	Feature definitions or UI formatting.
pairs	Curated universe, pair metadata, lifecycle inputs.	Cointegration calculations.
econometrics	Relationship estimation, diagnostics, spread state.	ML training or agent logic.
events	Divergence detection, event identity, labels.	Model fitting.
features	Point-in-time feature computation and schema.	UI or policy decisions.
ml	Training, calibration, inference, SHAP, model loading.	Data ingestion or Streamlit.
intelligence	Document cleaning, chunking, retrieval, structured catalyst extraction.	Final decision policy.
agents	LangGraph state, agent roles, output validation.	Persistent market-data writes.
decision	Deterministic gates and CANDIDATE/REVIEW/NO TRADE/ABSTAIN.	Free-form LLM reasoning.
backtest	Historical execution simulation, costs, metrics, ablations.	Production order placement.
services	Use-case orchestration for app/scripts.	Low-level mathematical implementations.
common	Shared IDs, time helpers, enums, exceptions, logging utilities.	Feature-specific logic.

5. Dependency rules

Preventing a tightly coupled codebase

5.1 Allowed direction

```
app  --->  services  --->  domain packages  --->  storage/config/common
                  |
                  +----------------------> artifact loaders

Batch scripts ---> services/domain jobs ---> storage/config/common
```

5.2 Explicit rules

- Presentation code may import service interfaces and presentation helpers only.
- Domain packages may share typed domain objects/enums through common, but should not import Streamlit.
- Storage code knows how to persist objects/tables; it does not decide what a valid pair, label, prediction or decision means.
- Agent code consumes deterministic quantitative state passed to it. It does not recompute prices, z-scores or XGBoost probabilities.
- Decision code consumes validated outputs from econometrics, ML, agents and data-quality checks; it does not call external LLMs.
- Backtesting consumes frozen historical outputs or explicitly versioned recomputations; it does not silently call the live app layer.
- Imports should be acyclic. Cross-package orchestration belongs in services/jobs, not by making packages call each other arbitrarily.

5.3 Service contracts

Services return typed, presentation-ready domain DTOs (Pydantic models or dataclasses) such as ScannerRow, PairDetail, InvestigationResult, StrategyLabResult and MonitorStatus. These are stable contracts between backend logic and Streamlit without introducing an HTTP API.

6. Configuration architecture

One source of truth for every experimental choice

Config file	Examples of contents
universe.yaml	Ticker pairs, sector/sub-industry, eligibility dates, notes, pair-universe version.
data.yaml	Date range, source settings, raw/curated paths, calendars, timestamp rules.
econometrics.yaml	Formation window, refit cadence, cointegration threshold, z-score rules.
ml.yaml	Feature schema version, partitions, hyperparameter search space, calibration method.
agents.yaml	Provider/model alias, evidence window, prompt versions, timeouts, retry limits.
decision.yaml	Probability thresholds, break-risk gates, hard/soft warnings, policy version.
backtest.yaml	Entry timing, exits, transaction costs, borrow assumptions, portfolio limits.
app.yaml	Demo mode, cache TTLs, page defaults, display limits.

6.1 Secrets and environment

- `.env` is local only and never committed.
- `.env.example` lists required variable names without secret values.
- Secrets contain credentials/tokens only; research parameters belong in versioned YAML.
- Production/demo secrets are supplied through the deployment platform secret store.
- All configuration is parsed into typed settings at startup; missing or malformed required values fail fast.

6.2 Reproducible config identity

Every run writes a normalized configuration snapshot and a config hash. Dataset version, feature version, model version, prompt/graph version, decision-policy version and backtest configuration remain separable so one layer can change without obscuring the others.

7. Physical storage layout

Files, Parquet, DuckDB, documents and artifacts

```
data/
|-- reference/  # security_master, pair_universe, sector_etf_map
|-- raw/
|   |-- yahoo/prices/... and yahoo/actions/...
|   |-- cboe/vix/...
|   |-- sec/company_tickers/...; submissions/...; filings/...
|   `-- gdelt/...  # optional
|-- curated/
|   |-- market/prices_daily.parquet
|   |-- market/corporate_actions.parquet
|   |-- market/market_context_daily.parquet
|   |-- evidence/sec_filing_index.parquet
|   |-- evidence/document_chunks.parquet
|   `-- evidence/news_index.parquet  # optional
|-- analytics/
|   `-- structural_break.duckdb
|-- snapshots/{data_snapshot_id}/manifest.json
|-- cache/{dashboard,agent_runs}/
`-- demo/{structural_break_demo.duckdb,...}

artifacts/
|-- models/
|-- calibrators/
|-- shap/
|-- manifests/
|-- reports/
`-- mlflow/
```

7.1 Storage responsibility

Store	Best suited for	Design rule
Raw files	Immutable source downloads and original documents.	Never overwrite silently; preserve source provenance.
Parquet	Large versioned tabular/time-series datasets.	Prefer append/new snapshot over in-place mutation.
DuckDB	Queryable analytical entities, metadata, decisions, run registries and views.	Batch writes; app reads.
Document store	SEC filing HTML/text and optional news bodies.	DB stores metadata/path, not large repeated document blobs.
Artifacts	Models, calibrators, SHAP references, manifests, reports.	Immutable versioned directories.
Cache	Derived demo outputs that can be regenerated.	Safe to delete; never treated as canonical evidence.

8. DuckDB implementation

Database organisation and access discipline

8.1 One project database

Use a single local analytical database at data/analytics/structural_break.duckdb. It is not an operational transaction database; it is an embedded research catalogue for joining pair state, events, predictions, evidence, decisions, trades and run metadata.

Logical schema	Contents
ref	Security/pair metadata and document metadata needed for joins.
market	Read-only views over Parquet market and market-context datasets.
analytics	Pair snapshots, divergence events, features, predictions, extractions, agent outputs, decisions, backtest trades, experiment results.
ops	Dataset versions, schema version, job runs, source health, artifact registry, refresh timestamps.

8.2 Why Parquet remains outside DuckDB

- Price history and feature snapshots are naturally columnar and can be versioned as files.
- DuckDB can query Parquet directly through views, avoiding unnecessary duplication.
- Large datasets can be replaced by a new versioned snapshot without rewriting unrelated analytical tables.
- The database stays small enough to copy for a demo while raw/curated files remain independently auditable.

8.3 Writer policy

Only explicit pipeline jobs write canonical tables. The Streamlit app opens read-only connections for ordinary pages. A completed job writes atomically where practical, records its job_run_id, updates version metadata, then refreshes dependent views/cache. Failed jobs do not partially redefine the current canonical snapshot.

9. Schema initialisation and migrations

Keeping database structure reproducible

```
sql/
|-- 001_schemas.sql
|-- 002_reference_tables.sql
|-- 003_analytics_tables.sql
|-- 004_ops_tables.sql
`-- 005_views.sql
```

9.1 Migration model

- Each SQL file is immutable after merge; later changes add a new numbered migration.
- An `ops.schema_migrations` table records applied migration number, checksum and timestamp.
- `scripts/init_db.py` creates a fresh database by applying migrations in order.
- `scripts/migrate_db.py` applies only unapplied migrations and refuses checksum mismatches.
- A fresh clone plus documented data snapshot must be able to rebuild the same schema without manual SQL edits.

9.2 Access modules

Infrastructure object	Responsibility
DuckDBConnectionFactory	Resolve database path, open connection, enforce read-only/read-write mode.
ParquetStore	Resolve versioned datasets and expose read/write helpers.
DocumentStore	Resolve immutable filing/document paths and metadata.
ArtifactStore	Resolve model/calibrator/SHAP/report directories and manifests.
RunRegistry	Write job/experiment/version metadata after successful operations.

> **No ORM required.** DuckDB is an analytical store and the project already works with explicit table schemas. Thin SQL/query modules are clearer than introducing SQLAlchemy or an ORM solely for abstraction.

10. Artifacts, manifests and version identity

Making every displayed result traceable

10.1 Artifact directory pattern

```
artifacts/models/model_A_v003/
|-- xgboost_model.json
|-- calibrator.joblib
|-- feature_schema.json
|-- training_manifest.json
|-- validation_metrics.json
|-- shap_background.parquet
`-- model_card.md
```

10.2 Run manifest

Manifest field	Purpose
run_id	Unique identifier for one job/experiment invocation.
git_commit	Exact source revision.
config_hash	Normalized configuration snapshot identity.
dataset_version	Exact input data snapshot.
pair_universe_version	Curated universe identity.
feature_schema_version	Predictor definitions/order.
model_version	Model + calibrator identity.
prompt_graph_version	Prompt templates and LangGraph state version.
policy_version	Decision controller thresholds/rules.
backtest_config_version	Execution/cost/portfolio assumptions.
created_at	UTC creation timestamp.

The app should display at least dataset as-of time, model version and policy version. The Methodology/Model Card page can expose the full manifest for reproducibility.

11. Pipeline jobs

Authoritative build graph

Job	Consumes	Produces
init_db	SQL migrations	DuckDB schema + version tables.
ingest_market_data	Universe + date config	Raw/curated market Parquet + source run record.
ingest_documents	Ticker/CIK/date config	Raw document store + document metadata.
build_pair_snapshots	Prices + pair config	Point-in-time relationship snapshots.
generate_events	Pair snapshots + event policy	Non-overlapping divergence events + labels.
build_features	Events + historical state	Versioned feature table.
train_model	Training/validation events/features	Model, calibrator, SHAP ref, metrics, manifest.
run_agent_batch	Events + admissible documents + model state	Structured extractions, agent outputs, break risk.
run_decision_batch	Validated state + policy	Final statuses + reason codes.
run_backtest	Frozen events/predictions/decisions	Ladder results, trades, metrics, report artifacts.
refresh_dashboard_cache	Canonical analytics	Fast demo-ready tables/files.

11.1 Orchestration

For the course build, use explicit Python scripts or a single `run_pipeline.py` orchestrator rather than Airflow/Prefect. The pipeline order is deterministic, small, and executed on demand. Each job must be independently rerunnable from versioned inputs.

> **Idempotency target.** Rerunning a job with the same code, config and input snapshot should either reproduce the same output version or detect that the output already exists. Jobs must not create silent duplicate events, predictions or decisions.

12. Application service layer

The boundary that replaces an API

The service layer is the only backend interface Streamlit needs. It gathers data from storage and domain modules, returns typed objects, and hides internal table joins/model-loading details from page code.

Service	Primary methods / use cases
ScannerService	Get current/historical candidate rows, apply filters, expose health/as-of metadata.
PairService	Load pair history, relationship diagnostics, event history, ML outputs, SHAP summaries.
InvestigationService	Load or execute bounded evidence retrieval + agent investigation for one event.
DecisionService	Load deterministic gate states and final decision explanation.
StrategyLabService	Load backtest ladder, ablations, robustness slices, rejected-trade analysis.
MonitorService	Dataset/model/source freshness, drift summaries, failed jobs, abstention/agent failure rates.
MethodologyService	Load active config/manifests/model card and project limitations.

12.1 Service design rules

- Services return stable typed DTOs instead of exposing raw DuckDB cursors or DataFrames directly to every page.
- Services are deterministic for a fixed repository state except explicitly marked on-demand agent calls.
- Pages do not know database table names or artifact paths.
- Services may compose multiple repositories/domain modules, but heavy computation should already be precomputed when possible.
- Service methods are unit/integration testable without launching Streamlit.

13. Streamlit application architecture

A polished UI without backend logic in pages

```
app/
|-- Home.py
|-- pages/
|   |-- 01_Scanner.py
|   |-- 02_Pair_Detail.py
|   |-- 03_Strategy_Lab.py
|   |-- 04_Model_Data_Monitor.py
|   `-- 05_Methodology_Model_Card.py
`-- components/
    |-- badges.py
    |-- charts.py
    |-- tables.py
    |-- filters.py
    `-- layout.py
```

13.1 Page responsibilities

Page	Backend service	Persistent writes?
Scanner	ScannerService	No.
Pair Detail	PairService + DecisionService	No.
Strategy Lab	StrategyLabService	No.
Model & Data Monitor	MonitorService	No.
Methodology / Model Card	MethodologyService	No.
Optional Investigate action	InvestigationService	Cache only unless later promoted by batch job.

13.2 Session state

`st.session_state` stores navigation/filter selections and temporary interaction state only: selected event ID, date, sector filters, expanded evidence panels, and whether a fresh investigation is being displayed. It is not the system of record for model outputs or research decisions.

> **App last, integration earlier.** A minimal shell can be created once Model A and pair-detail queries exist. Full styling, error states, agent evidence presentation, Strategy Lab polish and deployment occur near the end. This catches integration problems without letting UI work dominate research development.

14. Caching and performance

Keeping the demo responsive and deterministic

Cache target	Mechanism	Invalidation key
Read-only database handle	`st.cache_resource` or singleton service resource.	Database path + app process.
Small analytical queries	`st.cache_data`.	Query inputs + dataset/model/policy version.
Loaded XGBoost/calibrator/SHAP objects	Resource cache.	Artifact/model version.
Agent investigation output	Persistent file/cache entry.	event_id + source snapshot + prompt/graph + LLM config.
Dashboard aggregates	Precomputed batch cache.	analytics snapshot/version.
Large Plotly-ready series	Query/result cache.	pair/event + as-of/version.

14.1 Never cache by accident

- Every cached scientific result must include the versions that determine it.
- Do not cache a query only by ticker/event if the model or policy can change.
- Demo mode should prefer precomputed historical outputs over external calls.
- Cache failures/timeouts separately from valid agent conclusions; a failed call must not masquerade as a valid UNKNOWN classification.
- Cache can be cleared without damaging canonical research state.

15. Logging, errors and operational status

Making failures visible rather than silent

15.1 Structured run logging

Use standard Python logging with consistent context fields rather than ad-hoc print statements. Every batch job should log `run_id`, job name, dataset/config version, stage, record counts, duration, warnings, and final status. Sensitive tokens and full secret-bearing request payloads are never logged.

Error class	Expected behaviour
Configuration error	Fail immediately before a job begins.
Source/network error	Retry within bounded policy; record source failure; preserve previous canonical snapshot.
Schema/data-contract error	Fail job; do not publish partially validated dataset.
Econometric/model error on one pair/event	Record row-level failure/reason and continue only when scientifically defensible.
Agent invalid output	Reject output, retry within limit, otherwise UNKNOWN/ABSTAIN.
App query error	Show controlled error message and health context; never raw stack trace to end user.
Artifact/version mismatch	Fail closed; do not combine incompatible feature/model/policy versions.

15.2 Ops tables

- `ops.job_runs`: start/end/status/config hash/record counts/error summary.
- `ops.source_health`: last successful market/document refresh and status.
- `ops.artifact_registry`: model/report path, checksum, version and originating run.
- `ops.dataset_versions`: raw/curated snapshot identities and lineage.

16. Testing architecture

Tests mirror the scientific risk of the system

```
tests/
|-- unit/
|-- integration/
|-- leakage/
|-- regression/
|-- app/
`-- fixtures/
    |-- tiny_market/
    |-- tiny_documents/
    |-- synthetic_pairs/
    `-- mock_agent_outputs/
```

Suite	What it protects
unit	Deterministic functions: IDs, transforms, event rules, policy gates, storage helpers.
integration	Cross-layer flows: prices -> snapshot -> event -> features -> prediction; evidence -> agents -> decision.
leakage	No future timestamps, trailing-window correctness, embargo, historical document admissibility.
regression	Known fixture produces stable event counts, predictions, decisions and backtest summaries after refactors.
app smoke	Each page imports and renders against cached fixtures without internet or LLM access.

16.1 CI-safe fixtures

CI must not depend on yfinance, SEC availability, external LLMs or network access. Small committed fixtures represent known market series, documents and agent outputs. Network-dependent end-to-end refreshes are run manually or in a separate non-blocking workflow.

> **Highest-priority automated checks.** Point-in-time leakage tests and training/inference feature-schema consistency are release blockers. A visually correct app with leakage or feature-order mismatch is not an acceptable build.

17. Continuous integration and code quality

What every pull request must pass

17.1 GitHub Actions workflow

Step	Command / intent
Checkout + Python	Use pinned Python 3.11 environment.
Install	`uv sync --frozen` from committed lockfile.
Lint	Ruff checks import/style/static issues.
Types	mypy over `src/` and service interfaces.
Unit tests	Fast deterministic suite.
Leakage tests	Timestamp/window/partition invariants.
Integration tests	Small fixture pipeline.
App smoke	Import/render core pages in fixture/demo configuration.

17.2 PR gate

- No direct merge to main without passing CI.
- No generated large data/model artifacts committed to Git.
- Config/schema changes require corresponding test updates.
- Changes to event labels, feature definitions, model threshold, prompts, decision gates or backtest assumptions must increment the corresponding version identifier.
- If a change alters headline results, the pull request should link the affected experiment/run manifest.

18. Git and team collaboration

A group workflow that preserves research provenance

Element	Working convention
Main branch	Always runnable; protected; receives reviewed PRs only.
Feature branches	Short-lived, named by component/task rather than person.
Issues	One implementation task/bug per issue with acceptance criteria.
Pull requests	Small enough to review; describe data/model impact and test evidence.
Tags	Milestones such as `v0.1-data`, `v0.2-econ`, `v0.3-ml`, `v1.0-demo`.
Large artifacts	Stored outside Git; referenced by manifest/version/checksum.
Notebooks	Exploratory only; production logic must be moved into tested package modules.

18.1 Suggested ownership

Assign primary owners by subsystem (data/storage, econometrics/events, ML intelligence/agents, backtest/app) but keep interfaces documented so no layer becomes a single-person black box. Cross-review the leakage-sensitive and policy-sensitive modules.

18.2 Commit discipline

- Commit code/config changes separately from regenerated artifacts where possible.
- Do not commit `.env`, local DuckDB files, raw SEC documents, large Parquet datasets, MLflow run directories or cache files.
- Use deterministic scripts rather than manual spreadsheet edits to canonical datasets.

19. Environment and dependency management

Rebuildable on a clean machine

19.1 Runtime

Item	Specification
Python	3.11, pinned through `.python-version` and `pyproject.toml`.
Package manager	uv.
Dependency lock	Committed `uv.lock`; CI uses `--frozen`.
Project install	Package installed in editable/development mode locally.
Core stack	pandas/NumPy, DuckDB/Parquet, statsmodels, scikit-learn, XGBoost, SHAP, Pydantic/Pandera, LiteLLM, LangGraph, Streamlit, Plotly, MLflow, pytest, Ruff, mypy.
No FastAPI	No web backend framework or REST layer in the course build.

19.2 Clean-environment acceptance test

A new contributor should be able to clone the repository, install the locked environment, copy `.env.example` to `.env` if live credentials are needed, initialise the database, load either a frozen project snapshot or rebuild public data, run tests, and launch Streamlit using documented commands only.

19.3 Secrets

- LLM/API keys never appear in YAML, notebooks, test fixtures, logs or screenshots.
- The app must still run in demo mode without live external credentials.
- CI uses mocks/fixtures rather than production keys.

20. Deployment and demo mode

A reliable course submission rather than a fragile live system

20.1 Deployment shape

```
GitHub repository
    |
    +--> Streamlit Community Cloud (or approved equivalent)
         |
         +--> packaged source code
         +--> compact demo DuckDB / cached analytics
         +--> selected model artifacts
         `--> platform-managed secrets (optional live investigation)
```

20.2 Demo snapshot

- Bundle a compact, frozen analytics snapshot sufficient for Scanner, Pair Detail, Strategy Lab, Monitor and Methodology pages.
- Bundle the active model/calibrator/SHAP reference and model card.
- Precompute agent outputs for selected showcase events so the demo works without LLM connectivity.
- Keep raw historical market/SEC corpora out of the deployed package unless strictly needed.
- Expose a clear data-as-of timestamp and demo-mode badge when live refresh is disabled.

20.3 What deployment does not do

The deployed application does not schedule production ingestion, retrain models autonomously, write live orders, maintain a multi-user transaction database, or promise high availability. These would be production extensions outside the course scope.

21. Implementation sequence

How Codex should build the engineering foundation

Gate	Build work	Acceptance condition
E0 - repository	Create package skeleton, pyproject/uv, configs, logging, test folders, CI.	Clean install, imports succeed, lint/types/tests execute.
E1 - storage	Create directory resolver, DuckDB init/migrations, Parquet/document/artifact stores.	Fresh DB builds from migrations; read/write fixture test passes.
E2 - data integration	Wire ingestion jobs to storage/version registry.	Versioned snapshot reproducible from fixture/public source.
E3 - domain integration	Add econometrics/events/features/ML packages behind services.	End-to-end quantitative fixture path works.
E4 - app shell	Build Streamlit pages against Scanner/Pair services.	Scanner + Pair Detail render from cached fixture.
E5 - AI integration	Add intelligence/agents/decision services + cache.	Selected event returns validated structured investigation.
E6 - evaluation	Wire backtest/Strategy Lab/monitor services.	Full evaluation ladder loads from one versioned run.
E7 - polish	Demo snapshot, error states, model card, deployment, README/runbook.	Fresh deploy works without live API dependency.

Build philosophy. Create the repository and storage foundation first; develop backend layers incrementally; introduce a thin app shell once quantitative outputs exist; polish and deploy the app only after the research pipeline and evaluation are stable.

22. Definition of done

Engineering acceptance criteria

Area	Required condition
Repository	Clear package boundaries; no production logic trapped in notebooks or Streamlit pages.
Environment	Locked dependencies; clean setup documented and tested.
Configuration	All research parameters versioned; secrets external; startup validation.
Storage	Canonical raw/curated/analytics/artifact paths; DuckDB schema reproducible from migrations.
Database access	Batch writers and read-oriented app access separated; schema/version metadata recorded.
Services	Scanner, Pair Detail, Strategy Lab, Monitor and Methodology can be called without launching Streamlit.
App	Five core pages render from a frozen snapshot; no hidden model training inside callbacks.
Testing	Unit + integration + leakage + regression + app smoke suites pass.
CI	Ruff, mypy and deterministic tests pass on PRs without external network calls.
Reproducibility	Displayed result links to dataset/model/prompt/policy/backtest versions and Git commit.
Reliability	Demo mode works if data/LLM providers are unavailable.
Scope	No FastAPI, microservices, broker execution or unnecessary infrastructure added.

22.1 End-state engineering picture

A completed implementation should let a clean environment reproduce the research state, let batch jobs rebuild versioned layers in deterministic order, let tested services expose the resulting state, and let Streamlit present that state without owning scientific logic. The codebase should make it obvious where any displayed number, agent conclusion or backtest result came from and which versions produced it.